

OFFICIAL TECHNICAL DOCUMENTATION



RideMate AI

Intelligent Multi-Rider Shared Auto-Rickshaw Pooling & Real-Time Navigation Platform

Project Domain	Smart Urban Mobility & AI Routing
Frontend Architecture	Flutter (Dart) Cross-Platform Web & Mobile
Backend Engine	Node.js REST & Server-Sent Events (SSE)
Database Management	MariaDB / MySQL Relational Schema
Implementation Status	PRODUCTION READY

1. Executive Summary & Abstract

RideMate AI is an advanced, real-time shared mobility platform engineered to solve last-mile urban transit inefficiencies by digitalizing and synchronizing auto-rickshaw pooling. Urban commuters regularly encounter high solo fares, prolonged wait times, and traffic congestion. Concurrently, auto-rickshaws frequently operate with vacant seats, diminishing driver earnings and elevating per-passenger carbon emissions.

RideMate AI establishes a synchronized, consensual pooling workflow:

- **Algorithmic Geospatial Matching:** Utilizes spherical Haversine mathematics to detect riders traveling along overlapping corridors within a 2.5 km threshold.
- **Pre-Ride Verification & In-App Chat:** Allows matched commuters to communicate and verify mutual pickup points before any driver dispatch occurs.
- **Automated 50-50 Fare Splitting:** Calculates dynamic fair shares with transparent pricing.
- **Unified Single-Tap Driver Acceptance:** Binds both riders into a single pooled trip request accepted by the nearest driver in a single tap.
- **Smart Multi-Stop Navigation:** Directs drivers via optimal sequence (*Driver* → *Nearest Pickup* → *Second Pickup* → *Drop 1* → *Drop 2*) with 4-digit OTP security.

Core Innovation

Unlike traditional carpooling platforms that enforce blind, non-consensual matching with strangers, RideMate AI empowers commuters with pre-ride communication and mutual consent, establishing trust while delivering up to 50% fare savings.

2. Problem Statement & Motivation

2.1 The Urban Commuting Dilemma

In high-density educational and industrial hubs (e.g. Pune, Bangalore, Delhi NCR), millions of commuters travel along identical daily routes (such as metro stations to college campuses or tech parks). Solo hailing costs ₹120–₹160 per trip, while informal offline pooling causes arguments over fair division, chaotic stops, and safety concerns.

2.2 Key Project Objectives

- **Affordability:** Cut daily transit costs by up to 50% per passenger.
- **Driver Profitability:** Boost driver gross earnings per trip by 30–40% compared to empty single-fare runs.
- **Environmental Sustainability:** Reduce vehicle trips and urban carbon footprint through optimized shared occupancy.
- **Safety & Transparency:** Implement verified user badges, in-app messaging, and dual OTP trip authorization.

3. System Architecture & Lifecycle Flow

The platform employs a modular client-server architecture. The cross-platform Flutter frontend interfaces with a high-throughput Node.js microservice backed by an ACID-compliant MariaDB relational database.

- 1

Rider Discovery: Rider selects pickup & drop-off pins on interactive OpenStreetMap.
- 2

Co-Rider Matching: Backend matching engine scans active rides within 2.5 km corridor.
- 3

Pre-Ride Chat: In-app messaging room allows commuters to coordinate exact pickup spots.
- 4

Pool Confirmation: "Share This Ride" unifies both requests into a shared group.
- 5

Driver Dispatch & Live Route: Driver accepts joint request; navigation auto-frames multi-stop path.
- 6

OTP Start & Completion: Driver verifies 4-digit OTP to start trip and marks completed upon arrival.

4. Database Schema & Data Dictionary

The database schema is structured for transactional integrity, fast spatial queries, and concurrent multi-rider updates:

Table	Primary Key	Foreign Keys	Purpose
users	id (UUID)	—	Stores credentials, salted password hashes, phone numbers, and roles (RIDER , DRIVER).
driver_profiles	id (UUID)	user_id → users.id	Tracks vehicle type (Auto/E-Rickshaw), plate number, online state, seats, and live GPS coordinates.
rides	id (UUID)	rider_id , driver_id → users.id	Core transaction ledger with coordinates, fare shares, OTPs, pool group IDs, and statuses.
corider_requests	id (UUID)	sender_ride_id , receiver_ride_id	Manages pending and accepted co-rider connection invitations between commuters.
chat_messages	id (UUID)	sender_id → users.id	Persists real-time pre-ride conversations indexed by pool_group_id .

5. Technical Specifications & Algorithms

5.1 Geospatial Distance Calculation (Haversine Formula)

To guarantee sub-millisecond route proximity matching without external GIS API latency, the platform executes an in-memory spherical Haversine formula:

```
function calculateDistanceKm(lat1, lon1, lat2, lon2) {  
  const R = 6371; // Earth radius in kilometers  
  const dLat = (lat2 - lat1) * (Math.PI / 180);  
  const dLon = (lon2 - lon1) * (Math.PI / 180);  
  const a = Math.sin(dLat / 2) * Math.sin(dLat / 2) +  
    Math.cos(lat1 * (Math.PI / 180)) * Math.cos(lat2 * (Math.PI / 180)) *  
    Math.sin(dLon / 2) * Math.sin(dLon / 2);  
  const c = 2 * Math.atan2(Math.sqrt(a), Math.sqrt(1 - a));  
  return R * c;  
}
```

5.2 Multi-Stop Route Optimization

When a driver accepts a shared ride containing multiple commuters, the route sequencer determines the pick-up order dynamically:

1. Computes distance from Driver's GPS to Rider 1 Pickup (\$d_1\$) and Rider 2 Pickup (\$d_2\$).
2. Sets the nearest passenger as the **First Stop**.
3. Draws sequential navigation line: *Driver* → *Pickup 1* → *Pickup 2* → *Drop 1* → *Drop 2*.
4. Applies `CameraFit.bounds` to frame all active stops within the screen viewport automatically.

6. REST API Reference

Method	Endpoint	Auth	Description
POST	/api/auth/signup	Public	Registers a new rider or driver profile.
POST	/api/auth/login	Public	Authenticates credentials, clears stale rides, returns JWT token.
POST	/api/auth/logout	Bearer JWT	Marks driver offline and completes active test sessions.
POST	/api/rides/request	Bearer JWT	Creates a new searching ride entry with pickup/drop pins.
POST	/api/matches/nearby-rides	Bearer JWT	Returns list of nearby co-riders within 2.5 km threshold.
POST	/api/rides/share_confirm	Bearer JWT	Unifies two rides into a single <code>pool_group_id</code> and splits fare.
GET	/api/drivers/board	Bearer JWT	Returns available pooled and solo requests for online drivers.
POST	/api/rides/group/:id/accept	Bearer JWT	Driver accepts joint ride group; transitions riders to <code>ACCEPTED</code> .
POST	/api/rides/:id/start	Bearer JWT	Validates 4-digit OTP and begins shared trip.

7. Verification & Quality Assurance

RideMate AI was verified across extensive multi-client concurrent execution scenarios:

Test Scenario	Expected Result	Status
Simultaneous Rider Booking	Both riders discover each other within 2 seconds via live polling.	PASSED
Pre-Ride In-App Chat	Messages deliver instantly between riders with timestamps.	PASSED
Shared Group Acceptance	Single driver tap transitions both riders to ACCEPTED simultaneously.	PASSED
Full Screen Map Rendering	Unobstructed 60fps pan/zoom gestures with zero widget rebuild lag.	PASSED
Driver Logout Cleanup	Stale rides are completed upon logout; clean board on subsequent login.	PASSED
OTP Trip Authentication	Invalid OTP is rejected; correct OTP starts the whole pooled group.	PASSED

8. Installation & Deployment Guide

1. Database Setup:

Start MySQL / MariaDB on localhost:3306 (e.g. via XAMPP). The backend automatically handles schema creation.

2. Launching Backend Service:

```
cd c:\Ride_Mate_AI\backend
npm install
node src/server.js
# Output: Connected to MySQL database: ridemate
# Output: RideMate AI running at http://localhost:3000
```

3. Launching Frontend Application:

```
cd c:\Ride_Mate_AI\frontend
flutter pub get
flutter run -d web-server --web-port=8080
# Access application at http://localhost:8080
```

9. Conclusion

RideMate AI provides a comprehensive, technically robust, and socially intuitive solution for urban shared mobility. By addressing user trust through pre-ride chat, passenger affordability through automated fare splitting, and driver route efficiency through smart multi-stop auto-framing, it delivers a scalable blueprint for sustainable last-mile transit.